

Informe de laboratorio 02

Tema: Ordenamiento: Insertion Sort, Selection Sort

Nota

Estudiante	Escuela	Asignatura
Capatinta Almanza Camilo B. A. ccapatinta@unsa.edu.pe	Escuela Profesional de Ingeniería de Sistemas	Lab. Análisis y Diseño de Algoritmos Semestre: IV Código: 1702231

Laboratorio	Tema	Duración
02	Ordenamiento: Insertion Sort, Selection Sort	04 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - B	10/09/2026	14/09/2026

1. Ejercicios/Problemas propuestos

1.1. Ejercicio 1

Antes de resolver los ejercicios propuestos, compilen y ejecuten en su computadora el código de Insertion Sort y Selection Sort mostrado en la Sección II. Cámbienlo por un arreglo de datos propio (mínimo 8 elementos) y verifiquen que el resultado ordenado sea correcto. Esto les ayudará a comprobar que entienden cómo funciona cada algoritmo antes de continuar.

Listing 1: Ejercicio 01: InsertionSort y SelectionSort con más datos

```
1 #include <iostream>
2 #include <chrono>
3 using namespace std;
4
5 void printArray(int arr[], int n, int paso) {
6     cout << "Paso " << paso << ": ";
7
8     for (int i = 0; i < n; i++)
9         cout << arr[i] << " ";
10
11     cout << endl;
12 }
13
14 void insertionSort(int arr[], int n) {
```

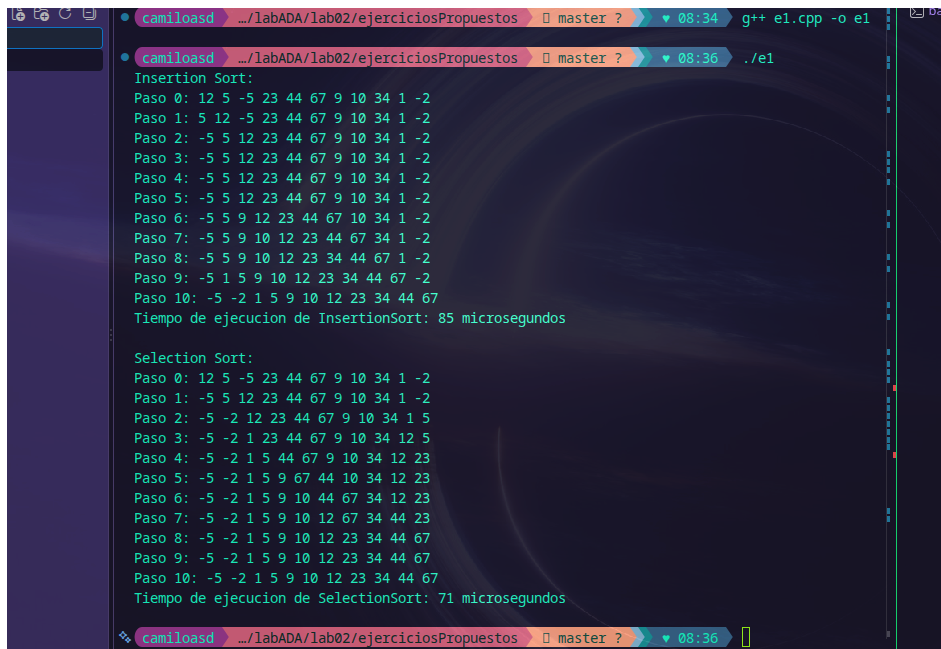
```
15     int paso = 0;
16
17     printArray(arr, n, paso++);
18
19     for (int i = 1; i < n; i++) {
20         int key = arr[i];
21         int j = i - 1;
22
23         while (j >= 0 && arr[j] > key) {
24             arr[j + 1] = arr[j];
25             j--;
26         }
27
28         arr[j + 1] = key;
29
30         printArray(arr, n, paso++);
31     }
32 }
33
34 void selectionSort(int arr[], int n) {
35     int paso = 0;
36
37     printArray(arr, n, paso++);
38
39     for (int i = 0; i < n - 1; i++) {
40         int minIdx = i;
41
42         for (int j = i + 1; j < n; j++) {
43             if (arr[j] < arr[minIdx])
44                 minIdx = j;
45         }
46
47         swap(arr[i], arr[minIdx]);
48
49         printArray(arr, n, paso++);
50     }
51 }
52
53 int main() {
54     int arrInsertion[] = {12, 5, -5, 23, 44, 67, 9, 10, 34, 1, -2};
55     int arrSelection[] = {12, 5, -5, 23, 44, 67, 9, 10, 34, 1, -2};
56     int n = sizeof(arrInsertion) / sizeof(arrInsertion[0]);
57
58     cout << "Insertion Sort:" << endl;
59
60     auto inicioInsertion = chrono::high_resolution_clock::now();
61     insertionSort(arrInsertion, n);
62     auto finInsertion = chrono::high_resolution_clock::now();
63
64     auto tiempoInsertion = chrono::duration_cast<chrono::microseconds>(finInsertion -
65         inicioInsertion);
66
67     cout << "Tiempo de ejecucion de InsertionSort: "
68         << tiempoInsertion.count()
69         << " microsegundos" << endl;
```

```

70     cout << "\nSelection Sort:" << endl;
71
72     auto inicioSelection = chrono::high_resolution_clock::now();
73     selectionSort(arrSelection, n);
74     auto finSelection = chrono::high_resolution_clock::now();
75
76     auto tiempoSelection = chrono::duration_cast<chrono::microseconds>(finSelection -
77         inicioSelection);
78
79     cout << "Tiempo de ejecucion de SelectionSort: "
80         << tiempoSelection.count()
81         << " microsegundos" << endl;
82
83     return 0;
84 }

```

- Se prueban InsertionSort y SelectionSort con un array de igual tamaño y de mismos datos, llegando ambos al mismo resultado pero con tiempos diferentes, siendo más eficiente SelectionSort porque demoró menos tiempo. Aunque esto puede cambiar en diferentes contextos y situaciones.



```

camiloasd .../labADA/lab02/ejerciciosPropuestos master ? 08:34 g++ e1.cpp -o e1
camiloasd .../labADA/lab02/ejerciciosPropuestos master ? 08:36 ./e1
Insertion Sort:
Paso 0: 12 5 -5 23 44 67 9 10 34 1 -2
Paso 1: 5 12 -5 23 44 67 9 10 34 1 -2
Paso 2: -5 5 12 23 44 67 9 10 34 1 -2
Paso 3: -5 5 12 23 44 67 9 10 34 1 -2
Paso 4: -5 5 12 23 44 67 9 10 34 1 -2
Paso 5: -5 5 12 23 44 67 9 10 34 1 -2
Paso 6: -5 5 9 12 23 44 67 10 34 1 -2
Paso 7: -5 5 9 10 12 23 44 67 34 1 -2
Paso 8: -5 5 9 10 12 23 34 44 67 1 -2
Paso 9: -5 1 5 9 10 12 23 34 44 67 -2
Paso 10: -5 -2 1 5 9 10 12 23 34 44 67
Tiempo de ejecucion de InsertionSort: 85 microsegundos

Selection Sort:
Paso 0: 12 5 -5 23 44 67 9 10 34 1 -2
Paso 1: -5 5 12 23 44 67 9 10 34 1 -2
Paso 2: -5 -2 12 23 44 67 9 10 34 1 5
Paso 3: -5 -2 1 23 44 67 9 10 34 12 5
Paso 4: -5 -2 1 5 44 67 9 10 34 12 23
Paso 5: -5 -2 1 5 9 67 44 10 34 12 23
Paso 6: -5 -2 1 5 9 10 44 67 34 12 23
Paso 7: -5 -2 1 5 9 10 12 67 34 44 23
Paso 8: -5 -2 1 5 9 10 12 23 34 44 67
Paso 9: -5 -2 1 5 9 10 12 23 34 44 67
Paso 10: -5 -2 1 5 9 10 12 23 34 44 67
Tiempo de ejecucion de SelectionSort: 71 microsegundos
camiloasd .../labADA/lab02/ejerciciosPropuestos master ? 08:36

```

Figura 1: Ejecución de e1.cpp

1.2. Ejercicio 2

Implementa el algoritmo Insertion Sort para ordenar una lista de n nombres ingresados por el usuario. Analiza su comportamiento con una lista ya ordenada, una lista en orden inverso y una lista aleatoria. ¿Cómo cambia el tiempo de ejecución?

Listing 2: Ejercicio 02: InsertionSort en una lista de n nombres y comparaciones con otras listas.

```

1 #include <iostream>

```

```
2 #include <vector>
3 #include <string>
4 #include <chrono>
5 using namespace std;
6
7 void insertionSortNombres(vector<string>& arr) {
8     int n = arr.size();
9     for (int i = 1; i < n; i++) {
10         string key = arr[i];
11         int j = i - 1;
12         while (j >= 0 && arr[j] > key) {
13             arr[j + 1] = arr[j];
14             j--;
15         }
16         arr[j + 1] = key;
17     }
18 }
19
20 int main() {
21     for (int iter = 1; iter <= 3; iter++) {
22         int n;
23         cout << "Ingrese la cantidad de elementos n para esta lista: ";
24         cin >> n;
25
26         vector<string> nombres(n);
27         cout << "Ingrese los " << n << " nombres (pueden ser ordenados, inversos o
                aleatorios):\n";
28         for (int i = 0; i < n; i++) {
29             cin >> nombres[i];
30         }
31
32         auto inicio = chrono::high_resolution_clock::now();
33         insertionSortNombres(nombres);
34         auto fin = chrono::high_resolution_clock::now();
35         chrono::duration<double, milli> duracion = fin - inicio;
36
37         cout << "\nLista ordenada:\n";
38         for (const string& nombre : nombres) {
39             cout << nombre << " ";
40         }
41         cout << "\n\nTiempo de ejecucion: " << duracion.count() << " ms\n";
42     }
43     return 0;
44 }
```

- Como el usuario ingresará datos, usamos `vector` para arrays dinámicos. En esencia el algoritmo se encuentra en las líneas 7 - 18, `insertionSortNombres(vector<string>& arr)` compara elementos y los reordena, desplazando elementos mayores hacia la izquierda para hacer espacio a los elementos que se colocarán en sus lugares.

```

camiloasd .../labADA/lab02/ejerciciosPropuestos master ? 11:13 g++ e2.cpp -o e2
camiloasd .../labADA/lab02/ejerciciosPropuestos master ? 11:13 ./e2
Ingrese la cantidad de elementos n para esta lista: 4
Ingrese los 4 nombres (pueden ser ordenados, inversos o aleatorios):
Abel
Bladimir
Camilo
Daniela

Lista ordenada:
Abel Bladimir Camilo Daniela

Tiempo de ejecucion: 0.015987 ms
Ingrese la cantidad de elementos n para esta lista: 4
Ingrese los 4 nombres (pueden ser ordenados, inversos o aleatorios):
Daniela
Camilo
Bladimir
Abel

Lista ordenada:
Abel Bladimir Camilo Daniela

Tiempo de ejecucion: 0.011779 ms
Ingrese la cantidad de elementos n para esta lista: 4
Ingrese los 4 nombres (pueden ser ordenados, inversos o aleatorios):
Bladimir
Daniel
Abel
Camilo

Lista ordenada:
Abel Bladimir Camilo Daniel

Tiempo de ejecucion: 0.0108 ms
camiloasd .../labADA/lab02/ejerciciosPropuestos master ? 11:14

```

Figura 2: Ejecución de e2.cpp

- Viendo la comparación de las 3 listas, quien demora menos tiempo es la lista desordenada, lo cual es contraintuitivo ya que una lista ordenada debería tomar menos tiempo. Esto es debido a que las condiciones del hardware pueden afectar, además las comparaciones son consecutivas, no simultáneas, por lo que para pequeños datos (3 strings) esta comparación no muestra realmente las complejidades temporales del algoritmo.

1.3. Ejercicio 3

Implementa el algoritmo Selection Sort sobre un arreglo de números flotantes. Muestra el número de comparaciones e intercambios realizados.

Listing 3: Ejercicio 03: SelectionSort sobre arreglo de numeros float.

```

1 #include <iostream>
2 #include <vector>
3
4 using namespace std;

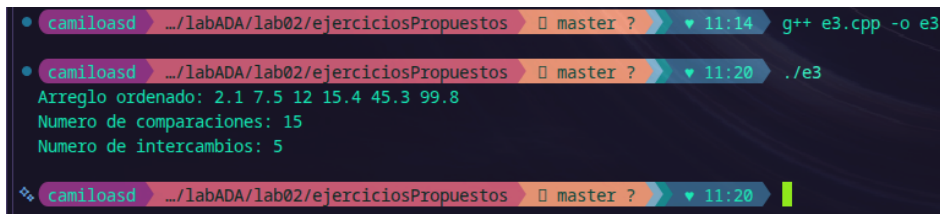
```

```

5
6 void selectionSortFloats(vector<float>& arr, int& comparaciones, int& intercambios) {
7     int n = arr.size();
8     comparaciones = 0;
9     intercambios = 0;
10
11     for (int i = 0; i < n - 1; i++) {
12         int minIdx = i;
13         for (int j = i + 1; j < n; j++) {
14             comparaciones++;
15             if (arr[j] < arr[minIdx]) {
16                 minIdx = j;
17             }
18         }
19         if (minIdx != i) {
20             swap(arr[i], arr[minIdx]);
21             intercambios++;
22         }
23     }
24 }
25
26 int main() {
27     vector<float> numeros = {15.4, 2.1, 99.8, 45.3, 12.0, 7.5};
28     int comp = 0, inter = 0;
29
30     selectionSortFloats(numeros, comp, inter);
31
32     cout << "Arreglo ordenado: ";
33     for (float f : numeros) cout << f << " ";
34     cout << "\nNumero de comparaciones: " << comp;
35     cout << "\nNumero de intercambios: " << inter << endl;
36
37     return 0;
38 }

```

- En las líneas 6 – 24, la función `selectionSortFloats` asume inicialmente que el primer elemento de la parte aún no ordenada es el mínimo (`minIdx = i`). Luego recorre los elementos restantes y, en cada comparación, actualiza `minIdx` si encuentra un valor menor. Al finalizar el recorrido, intercambia el mínimo encontrado con el elemento de la posición `i`, únicamente si están en posiciones diferentes. El programa usa contadores para registrar el número de comparaciones y de intercambios realizados.



```

camiloasd .../labADA/lab02/ejerciciosPropuestos master ? 11:14 g++ e3.cpp -o e3
camiloasd .../labADA/lab02/ejerciciosPropuestos master ? 11:20 ./e3
Arreglo ordenado: 2.1 7.5 12 15.4 45.3 99.8
Numero de comparaciones: 15
Numero de intercambios: 5
camiloasd .../labADA/lab02/ejerciciosPropuestos master ? 11:20

```

Figura 3: Ejecución de `e3.cpp`

1.4. Ejercicio 4

Crea una función que reciba un arreglo y un tipo de ordenamiento (insertion o selection) y aplique el algoritmo correspondiente. Evalúa el rendimiento con 1000, 5000 y 10000 elementos aleatorios.

Listing 4: Ejercicio 04: Función que recibe el tipo de ordenamiento.

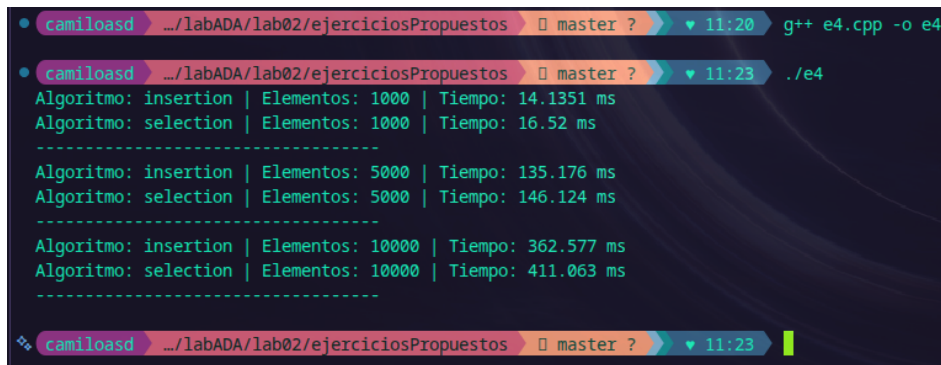
```
1  #include <iostream>
2  #include <vector>
3  #include <string>
4  #include <chrono>
5  #include <cstdlib>
6  #include <algorithm>
7
8  using namespace std;
9
10 void insertionSort(vector<int>& arr) {
11     int n = arr.size();
12     for (int i = 1; i < n; i++) {
13         int key = arr[i];
14         int j = i - 1;
15         while (j >= 0 && arr[j] > key) {
16             arr[j + 1] = arr[j];
17             j--;
18         }
19         arr[j + 1] = key;
20     }
21 }
22
23 void selectionSort(vector<int>& arr) {
24     int n = arr.size();
25     for (int i = 0; i < n - 1; i++) {
26         int minIdx = i;
27         for (int j = i + 1; j < n; j++) {
28             if (arr[j] < arr[minIdx]) {
29                 minIdx = j;
30             }
31         }
32         if (minIdx != i) {
33             swap(arr[i], arr[minIdx]);
34         }
35     }
36 }
37
38 void evaluarOrdenamiento(vector<int> arr, string tipo) {
39     auto inicio = chrono::high_resolution_clock::now();
40
41     if (tipo == "insertion") {
42         insertionSort(arr);
43     } else if (tipo == "selection") {
44         selectionSort(arr);
45     }
46
47     auto fin = chrono::high_resolution_clock::now();
48     chrono::duration<double, milli> duracion = fin - inicio;
49
50     cout << "Algoritmo: " << tipo << " | Elementos: " << arr.size()
```

```

51         << " | Tiempo: " << duracion.count() << " ms" << endl;
52     }
53
54     int main() {
55         vector<int> tamanios = {1000, 5000, 10000};
56
57         for (int tam : tamanios) {
58             vector<int> datos(tam);
59             generate(datos.begin(), datos.end(), []() { return rand() % 100000; });
60
61             evaluarOrdenamiento(datos, "insertion");
62             evaluarOrdenamiento(datos, "selection");
63             cout << "-----" << endl;
64         }
65
66         return 0;
67     }

```

- Se usan las librerías `cstdlib` y `algorithm` para generar y colocar números pseudoaleatorios en los vectores. En `main()` se crea un vector con los tamaños 1000, 5000 y 10 000. Para cada tamaño, se crea un vector de enteros y se llena con valores entre 0 y 99999. Después se llama a `evaluarOrdenamiento()` dos veces: una para Insertion Sort y otra para Selection Sort. Cada algoritmo recibe una copia del mismo vector original, permitiendo comparar sus tiempos de ejecución de manera justa.



```

camiloasd ~/labADA/lab02/ejerciciosPropuestos master ? 11:20 g++ e4.cpp -o e4
camiloasd ~/labADA/lab02/ejerciciosPropuestos master ? 11:23 ./e4
Algoritmo: insertion | Elementos: 1000 | Tiempo: 14.1351 ms
Algoritmo: selection | Elementos: 1000 | Tiempo: 16.52 ms
-----
Algoritmo: insertion | Elementos: 5000 | Tiempo: 135.176 ms
Algoritmo: selection | Elementos: 5000 | Tiempo: 146.124 ms
-----
Algoritmo: insertion | Elementos: 10000 | Tiempo: 362.577 ms
Algoritmo: selection | Elementos: 10000 | Tiempo: 411.063 ms
-----
camiloasd ~/labADA/lab02/ejerciciosPropuestos master ? 11:23

```

Figura 4: Ejecución de e4.cpp

1.5. Ejercicio 5

Desarrollen una aplicación en C++ que permita ordenar una lista de elementos sobre una temática libre elegida por el estudiante (en este caso, libros). Cada elemento debe contener al menos los siguientes atributos:

- Un identificador o código (string o entero).
- Un nombre o título (string).
- Un valor numérico que sirva como criterio de ordenamiento (número de páginas).

Pasos a seguir:

- Definir la temática y la estructura de los datos.

- Cargar los datos dinámicamente desde un archivo `.txt`.
- Ordenar los elementos de mayor a menor según el valor numérico, usando Insertion Sort y Selection Sort.
- Implementar contadores para las comparaciones realizadas y los intercambios efectuados.
- Medir el tiempo de ejecución.
- Exportar la lista ordenada a un archivo `.csv`.

Listing 5: Ejercicio 05: Creación de aplicación que ordena una lista de libros por número de páginas.

```
1 #include <iostream>
2 #include <vector>
3 #include <string>
4 #include <fstream>
5 #include <sstream>
6 #include <chrono>
7
8 using namespace std;
9
10 struct Libro {
11     string codigo;
12     string titulo;
13     int paginas;
14 };
15
16 void cargarDatos(const string& filename, vector<Libro>& lista) {
17     ifstream archivo(filename);
18     string linea;
19     while (getline(archivo, linea)) {
20         stringstream ss(linea);
21         string cod, nom, pagStr;
22         if (getline(ss, cod, ',') && getline(ss, nom, ',') && getline(ss, pagStr, ',')) {
23             lista.push_back({cod, nom, stoi(pagStr)});
24         }
25     }
26 }
27
28 void insertionSortDesc(vector<Libro>& arr, long long& comp, long long& inter) {
29     int n = arr.size();
30     comp = 0; inter = 0;
31     for (int i = 1; i < n; i++) {
32         Libro key = arr[i];
33         int j = i - 1;
34         while (j >= 0) {
35             comp++;
36             if (arr[j].paginas < key.paginas) {
37                 arr[j + 1] = arr[j];
38                 inter++;
39                 j--;
40             } else {
41                 break;
42             }
43         }
44         arr[j + 1] = key;
```

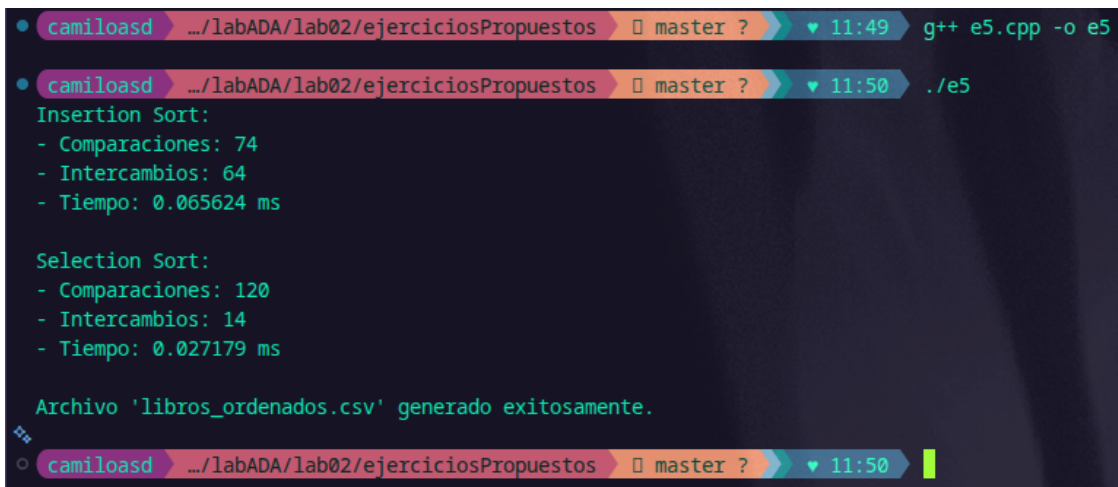
```
45     }
46 }
47
48 void selectionSortDesc(vector<Libro>& arr, long long& comp, long long& inter) {
49     int n = arr.size();
50     comp = 0; inter = 0;
51     for (int i = 0; i < n - 1; i++) {
52         int maxIdx = i;
53         for (int j = i + 1; j < n; j++) {
54             comp++;
55             if (arr[j].paginas > arr[maxIdx].paginas) {
56                 maxIdx = j;
57             }
58         }
59         if (maxIdx != i) {
60             swap(arr[i], arr[maxIdx]);
61             inter++;
62         }
63     }
64 }
65
66 void exportarCSV(const string& filename, const vector<Libro>& lista) {
67     ofstream archivo(filename);
68     archivo << "Codigo,Titulo,Paginas\n";
69     for (const auto& l : lista) {
70         archivo << l.codigo << "," << l.titulo << "," << l.paginas << "\n";
71     }
72 }
73
74 int main() {
75     vector<Libro> libros;
76     cargarDatos("libros.txt", libros);
77
78     if (libros.empty()) {
79         cout << "Error: No se encontraron datos en libros.txt" << endl;
80         return 1;
81     }
82
83     vector<Libro> librosIns = libros;
84     vector<Libro> librosSel = libros;
85
86     long long compIns = 0, interIns = 0;
87     long long compSel = 0, interSel = 0;
88
89     auto t1 = chrono::high_resolution_clock::now();
90     insertionSortDesc(librosIns, compIns, interIns);
91     auto t2 = chrono::high_resolution_clock::now();
92     double tiempoIns = chrono::duration<double, milli>(t2 - t1).count();
93
94     t1 = chrono::high_resolution_clock::now();
95     selectionSortDesc(librosSel, compSel, interSel);
96     t2 = chrono::high_resolution_clock::now();
97     double tiempoSel = chrono::duration<double, milli>(t2 - t1).count();
98
99     cout << "Insertion Sort:\n";
100    cout << "- Comparaciones: " << compIns << "\n";
```

```

101 cout << "- Intercambios: " << interIns << "\n";
102 cout << "- Tiempo: " << tiempoIns << " ms\n\n";
103
104 cout << "Selection Sort:\n";
105 cout << "- Comparaciones: " << compSel << "\n";
106 cout << "- Intercambios: " << interSel << "\n";
107 cout << "- Tiempo: " << tiempoSel << " ms\n";
108
109 exportarCSV("libros_ordenados.csv", librosIns);
110 cout << "\nArchivo 'libros_ordenados.csv' generado exitosamente.\n";
111
112 return 0;
113 }

```

- Lee los libros almacenados en libros.txt, obteniendo de cada uno su código, título y cantidad de páginas. Luego crea dos copias de la misma lista para ordenarlas de forma descendente según el número de páginas: una mediante Insertion Sort y otra mediante Selection Sort. Durante cada ordenamiento, registra la cantidad de comparaciones, los movimientos o intercambios realizados y el tiempo de ejecución. Después muestra estos resultados en pantalla para comparar ambos algoritmos y, finalmente, guarda la lista ordenada por Insertion Sort en el archivo libros_ordenados.csv.



```

camiloasd ~/labADA/lab02/ejerciciosPropuestos master ? 11:49 g++ e5.cpp -o e5
camiloasd ~/labADA/lab02/ejerciciosPropuestos master ? 11:50 ./e5
Insertion Sort:
- Comparaciones: 74
- Intercambios: 64
- Tiempo: 0.065624 ms

Selection Sort:
- Comparaciones: 120
- Intercambios: 14
- Tiempo: 0.027179 ms

Archivo 'libros_ordenados.csv' generado exitosamente.
camiloasd ~/labADA/lab02/ejerciciosPropuestos master ? 11:50

```

Figura 5: Ejecución de e5.cpp

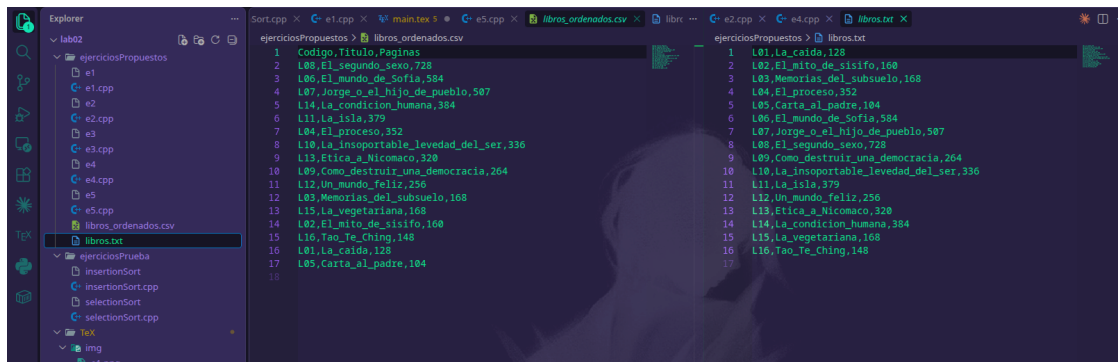


Figura 6: Comparación de .csv (ordenado por n° de páginas) vs .txt (desordenado por n° de páginas)

2. Cuestionario

1. Explica con tus propias palabras: ¿cuál de los dos algoritmos es más eficiente si la lista ya está ordenada? ¿Por qué? Usa un ejemplo distinto al visto en clase.

Si la lista ya está ordenada, Insertion Sort es más eficiente. Esto se debe a que, al revisar cada elemento, observa que ya está en la posición correcta y no necesita desplazar elementos. Por ejemplo, si se tiene la lista de edades {18, 20, 22, 25, 30}, Insertion Sort solo compara cada edad con la anterior y continúa, porque no encuentra ningún valor que deba mover. En cambio, Selection Sort sigue buscando el menor elemento en toda la parte no ordenada, aunque los datos ya estén ordenados.

2. En tus palabras, ¿qué significa que un algoritmo sea *in-place*? Da un ejemplo de tu vida cotidiana donde "ordenar sin usar espacio extra" tendría sentido.

Que un algoritmo sea *in-place* significa que ordena los datos usando muy poca memoria adicional, normalmente sin crear otra lista del mismo tamaño. Por ejemplo, al ordenar libros en un estante, se puede tomar un libro y cambiarlo de posición con otro dentro del mismo estante, sin necesitar una segunda estantería para guardar todos los libros temporalmente.

3. ¿Por qué se dice que Selection Sort no es estable? Describe un caso concreto (puede ser inventado por ti) donde eso cause un problema real.

Se dice que Selection Sort no es estable porque puede cambiar el orden original de dos elementos que tienen la misma clave de comparación. Por ejemplo, si se ordenan libros por cantidad de páginas y se tienen los libros A(200), B(150) y C(200), al hacer un intercambio, los libros A y C, que tienen la misma cantidad de páginas, podrían quedar en un orden diferente al original. Esto sería un problema si el orden inicial representara otro criterio importante, como la fecha de publicación o el orden en que los libros fueron registrados.

4. ¿Qué parte te costó más entender entre Insertion Sort y Selection Sort, y cómo la resolviste?

La parte que más me costó entender fue la diferencia entre los desplazamientos de Insertion Sort y los intercambios de Selection Sort. Lo resolví siguiendo manualmente una lista pequeña y observando qué ocurría en cada iteración. Así comprendí que Insertion Sort mueve elementos para insertar uno en su posición correcta, mientras que Selection Sort busca el menor o mayor elemento y realiza un intercambio al final de cada pasada.

5. Si tuvieras que explicarle a un compañero que faltó a la práctica cómo optimizar uno de estos dos algoritmos, ¿qué le dirías?

Para optimizar Insertion Sort, le diría a un compañero que es conveniente usarlo cuando la lista está ordenada o casi ordenada, porque en esos casos realiza pocos desplazamientos y puede ser rápido. Sin embargo, para listas muy grandes y desordenadas, recomendaría utilizar algoritmos más eficientes, como Merge Sort o Quick Sort, ya que Insertion Sort y Selection Sort tienen un tiempo de ejecución aproximado de $O(n^2)$.

3. Conclusiones

- Se comprobó que Insertion Sort y Selection Sort permiten ordenar datos correctamente, pero su rendimiento puede variar según el orden inicial de los elementos.
- Insertion Sort resulta más conveniente cuando los datos ya están ordenados o casi ordenados, debido a que requiere menos desplazamientos.
- Selection Sort realiza una cantidad fija de comparaciones para un mismo tamaño de lista, aunque esta ya se encuentre ordenada; sin embargo, normalmente realiza pocos intercambios.
- Ambos algoritmos tienen una complejidad aproximada de $O(n^2)$, por lo que no son los más recomendables para listas muy grandes.
- La práctica permitió comprender cómo medir comparaciones, movimientos e tiempo de ejecución para evaluar el comportamiento de un algoritmo de ordenamiento.